

Foguete.io: *Relatório do Trabalho Prático*

Programação Concorrente (2025/2026) - Licenciatura em Ciências da Computação, UMinho

Nome	Número
Ivo Costa Sousa	A102935
Ricardo Eusébio Cerqueira	A102878
Miguel Dinis Páscoa	A104244

1. Introdução

Este documento detalha o desenvolvimento do **Foguete.io**, uma aplicação interativa que concretiza os requisitos de um mini-jogo distribuído. O projeto assenta numa arquitetura cliente-servidor para simular um espaço 2D onde os avatares dos jogadores interagem entre si e com o ambiente envolvente em tempo real. O cliente, responsável pela interface gráfica e captura de inputs, foi escrito em **Java** através da biblioteca **Processing**. A lógica central e simulação física do jogo estão delegadas num servidor escrito em **Erlang**, cuja natureza permite uma gestão eficiente e concorrente de múltiplos jogadores. A troca de mensagens entre nós é realizada através de *sockets TCP*, regida por um protocolo textual orientado à linha construído no âmbito da unidade curricular.

2. Arquitetura do Sistema

2.1 Servidor (Erlang)

O servidor segue o modelo de **processos e passagem de mensagens**. É constituído pelos seguintes processos:

- **accept_loop**: aguarda novas ligações TCP e lança um processo por cliente.
- **gestor_de_contas**: gere o registo, login, logout e cancelamento de contas. Mantém em memória uma lista de utilizadores online para impedir sessões duplicadas.
- **gestor_de_scores**: mantém em memória (lista de tuplos {User, Pontos}) as pontuações máximas de cada jogador desde que o servidor foi arrancado.
- **gestor_de_partidas**: gere a fila de espera e as salas ativas. Inicia partidas com 3 ou 4 jogadores e suporta até 4 partidas simultâneas.
- **sala_de_jogo**: processo por partida, responsável pela simulação física, deteção de colisões, e envio do estado do jogo a todos os jogadores a cada ~16ms (60fps).

Cada cliente passa por três estados no servidor: `cliente_autenticacao` → `cliente_espera` → `cliente_receptor`.

2.2 Cliente (Java/Processing)

O cliente é constituído por duas threads:

- **Thread principal (Processing):** corre o `draw()` a 60fps, responsável por desenhar o estado do jogo.
- **Thread de rede (ouvinte):** lê mensagens do servidor e atualiza o estado partilhado.

O estado partilhado entre as duas threads é protegido por um **ReentrantLock**, garantindo exclusão mútua. O padrão `lock.lock() / finally { lock.unlock() }` é aplicado consistentemente em todas as secções críticas.

3. Concorrência em Java

3.1 Exclusão Mútua com ReentrantLock

A principal preocupação de concorrência no cliente é a partilha de estado entre a thread de rede e o `draw()`. Sem proteção, seria uma race condition idêntica ao problema do Counter do Guião 1.

O **ReentrantLock** protege todas as variáveis partilhadas: `estadoJogo`, `objetos`, `adversarios`, `nomesHighscores`, entre outros. O `draw()` adquire o lock no início e liberta-o no `finally`:

```
void draw() {
    lock.lock();
    try {
        switch (estadoJogo) { ... }
    } finally {
        lock.unlock();
    }
}
```

3.2 Double Buffer para o Estado do Jogo

Para evitar que o `draw()` veja um frame a meio de uma atualização, usamos buffers de espera (`objetosEspera`, `adversariosEspera`) que são trocados atomicamente dentro do lock quando o servidor envia a mensagem **SYNC**:

```
case "SYNC":
    objetos.clear();
    objetos.addAll(objetosEspera);
    adversarios.clear();
    adversarios.addAll(adversariosEspera);
```

```
adversariosNomes.clear();
adversariosNomes.addAll(adversariosNomesEspera);
break;
```

3.3 Variável de Condição

É utilizada uma `Condition` associada ao `ReentrantLock` para sinalizar mudanças de estado relevantes (login aceite, partida iniciada, resultados recebidos), evitando polling desnecessário.

4. Concorrência em Erlang

4.1 Estado em Memória

Todo o estado do servidor é mantido em memória como argumentos dos processos, sem variáveis globais. Por exemplo, o `gestor_de_contas` mantém a base de dados DETS para persistência de contas e uma lista em memória de utilizadores online:

```
gestor_de_contas(Db, Online) ->
    receive
        {login, User, Pass, FromPid} ->
            case lists:member(User, Online) of
                true  -> FromPid ! {resultado_login, erro_ja_online}, ...;
                false -> FromPid ! {resultado_login, sucesso}, ...
            end;
        {logout, User} ->
            gestor_de_contas(Db, lists:delete(User, Online))
    end.
```

As pontuações são mantidas exclusivamente em memória (não persistidas), pois o enunciado especifica que o top de scores é válido apenas desde que o servidor foi arrancado.

4.2 Loop de Simulação

A `sala_de_jogo` usa o `after 16` do `receive` para executar a simulação física a cada ~16ms, calculando movimento, fricção, colisões com objetos e entre jogadores, e enviando o estado atualizado a todos os clientes.

4.3 Comunicação TCP

O servidor usa `{packet, line}`, garantindo que cada `receive` entrega sempre uma linha completa ao processo, evitando fragmentação de mensagens TCP.

5. Protocolo de Comunicação

A comunicação usa um protocolo textual simples, com campos separados por vírgulas e terminados em \n:

Mensagem	Direção	Descrição
REGISTER,user,pass	C→S	Registo de conta
LOGIN,user,pass	C→S	Autenticação
LOGIN_OK	S→C	Login aceite
LOGIN_FAIL_ONLINE	S→C	Utilizador já em jogo
MATCH_START,tempoMs	S→C	Partida iniciada com tempo restante
KEYS,UP,LEFT,RIGHT	C→S	Input do jogador
STATE,user,x,y,tam,ang,kills	S→C	Estado do próprio jogador
OTHER,user,x,y,tam,ang	S→C	Estado de outro jogador
OBJ,x,y,tam,tipo	S→C	Objeto no espaço
SYNC	S→C	Fim de frame - troca de buffers
MATCH_RESULTS,u,pts,...	S→C	Resultados finais
GET_HIGHScores	C→S	Pedido de top de pontuações
HIGHScores,u,pts,...	S→C	Top 5 pontuações
BACK_TO_MENU	C→S	Jogador volta ao menu

6. Funcionalidades Implementadas

- **Registo, login, logout e cancelamento de conta** com persistência em DETS.
- **Prevenção de sessões duplicadas:** o mesmo utilizador não pode entrar em dois clientes em simultâneo.
- **Fila de espera** com arranque automático de partidas com 3 ou 4 jogadores.
- **Até 4 partidas simultâneas**, cada uma com o seu processo sala_de_jogo independente.
- **Simulação física** no servidor: movimento com aceleração linear e angular, fricção, colisões com bordas.
- **Colisões:** objetos venenosos penalizam o jogador; objetos comestíveis aumentam a massa; a captura de jogadores transfere massa e incrementa o contador de kills; jogadores com massas equivalentes sofrem repulsão mútua ao colidir, sendo impulsionados em direções opostas.
- **Pontuação** baseada no número de capturas de outros jogadores.
- **Top 5 de pontuações** visível no lobby e num ecrã dedicado.
- **Sincronização do timer:** jogadores que entram a meio de uma partida recebem o tempo restante correto do servidor.
- **Interface gráfica** com menu, lobby, jogo, resultados e ecrã de classificações.